

Individual Contribution Reflection

My primary responsibility was within frontend integration, component structure, end-to-end support for the image-based question variation. Notably I worked with Chong An (who went by the alias Tenko in his github) to flesh out the variation that we have decided: Image Url. Initial discussions with Aki also led us to our current UI choices. Essentially my role eventually evolved into making the frontend reliable across machines, making it readable and clean, and to complete the variation we have chosen.

Frontend UI

My earliest commits focused on UX and theme compatibility. When discussing with Aki, we decided he would provide a scaffold and I improved upon it. Hence I introduced styling for the login page (which the other team members proceeded to use as reference for the other pages), and utilising MUI's library to allow switches between light and dark mode.

Evidence: 2292aee / 884d629 (login styling), 58d2650 / 2906e49 (dark mode fixes).

System Integration

After Tenko merged the admin and image-variation branch, I focused on shared frontend-backend behaviour so the app worked consistently on our machines. I aligned the default API port and Vite proxy to 5001 (be30cc6), introduced handleApiResponse as the single entry point for all API modules (d25f747), and migrated layouts to the MUI v9 Grid API across affected pages (aed2f84). Together, these changes standardised how the client talks to the backend and how errors reach the UI.

Evidence: be30cc6, d25f747, aed2f84

Component Architecture

Besides that, I also attempted to improve code readability by dividing large page files that makes it risky to work on, and removing duplicates into dedicated components. I extracted PageLoader (duplicated code in many different files). I also separated Quiz and Admin pages into individual components, leaving pages to load data and call adminApi / quizApi only.

Image-Based Variation

Tenko's merge (4c9a491) introduced optional imageUrl on the model, admin forms, and quiz API responses. I extended that work so the variation could be loaded, checked, and demonstrated without manual database edits. On the backend (1d62784), I updated bulk import to trim and store imageUrl, expanded seed.js so most seeded questions include images (with npm run seed:reset for a clean reset), and added a Jest test that bulk import preserves imageUrl when provided. On the frontend (a2bd588), I updated BulkImport.jsx (placeholder JSON and docs for HTTPS and /images/... paths) and added an image column with thumbnails in AdminQuestionsTable.jsx. QuizQuestion.jsx (from 978b8f6) renders the image above the question text when present, with onError hiding broken URLs so the quiz flow continues.

Evidence: 1d62784, a2bd588

Key Technical Challenge

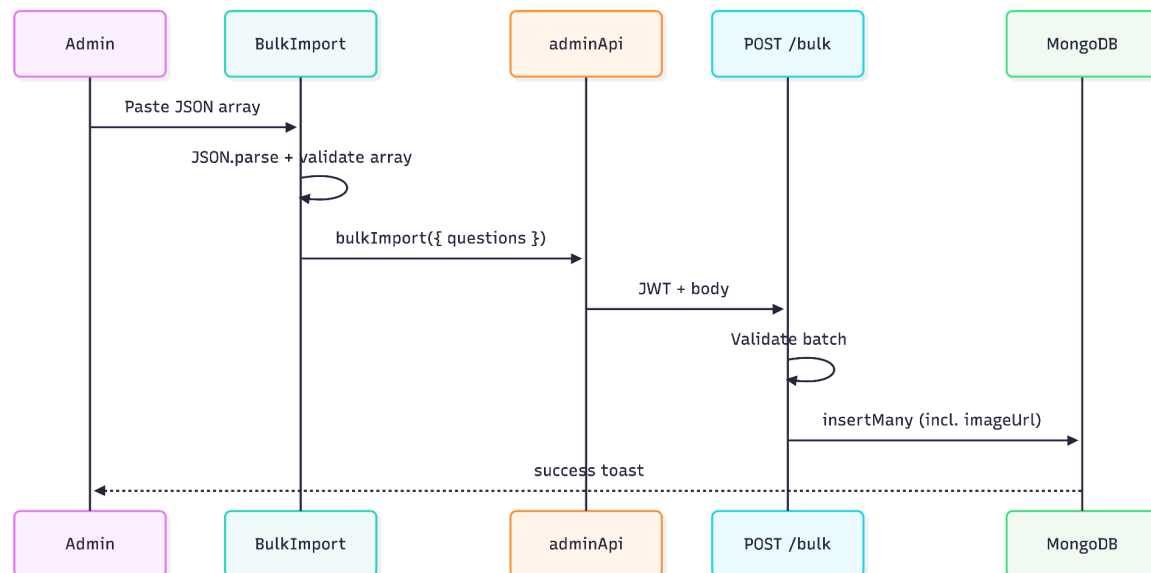
During development, one of the challenges came after Chong An merged the admin and image-variation branch. The reason for the challenge was not due to technical complexity, but more so because it was a niche problem. As mentioned earlier, the macOS bounds port 5000 onto AirPlay, which takes priority over our application. This caused errors where the network tab returned an empty body or non-JSON.

I fixed this in two linked steps. In be30cc6, I set the default backend port and Vite proxy to 5001 so frontend fetch("/api/...") and Express listen on the same port. In d25f747, I added `handleApiResponse` and routed `auth.js`, `quiz.js`, and `admin.js` through it. The helper reads the body as text, rejects empty responses, parses JSON safely, checks success, and returns data or throws a clear message (including a hint about port 5001 on macOS)

After my fix, any similar errors can be more actionable instead of having to stare at silent parse errors. The same path is used for admin bulk import and quiz loading.

Evidence: 7ee208c (PageLoader), 978b8f6 (QuizQuestion, AdminQuestionsTable, DeleteQuestionDialog), 7513006 (Home layout).

Diagram



In a group of three, my part was the frontend boundary: one response handler, aligned port/proxy, consistent Grid API, and client-side checks on bulk import before calling the API. Aki provided the backend envelope and validation patterns (e.g. validate-before-insert, Many on bulk); Chong An delivered the initial image variation scaffold. I then connected those pieces so behaviour and error messages stayed consistent across machines.